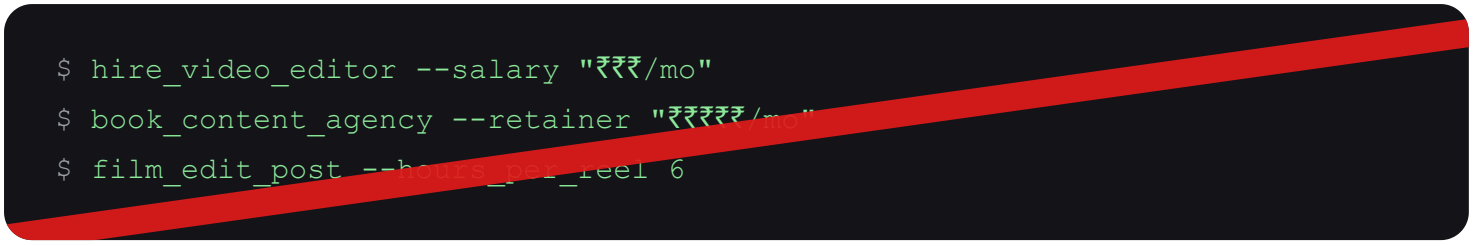
FREE GUIDE · @AUTOMATEWITHANAND

5 Claude Code Skills That Replaced My Content Team

no editor, no agency, no crew — the exact system I run my content business on



I fired all of it. I now run the whole thing — idea → script → avatar video → thumbnail → caption → posted — with **five Claude Code skills** that talk to my own build system. This guide hands you all five: what each one does, the exact `SKILL.md` you paste into Claude Code, the command to run it, and the traps that waste your first week.

Why skills, not prompts

A prompt is a one-time favour. You explain everything, get one answer, and start from zero next time. A **skill** is a saved instruction file (`SKILL.md`) that lives in your project. Claude Code loads it automatically, so the model already knows *your* voice, *your* format library, and *your* commands — every time, with no re-explaining. Prompts don't compound. Skills do. That's the whole difference between "I use AI sometimes" and "AI runs my content line."

Where a skill lives: create `.claude/skills/<skill-name>/SKILL.md` in your project. Claude Code auto-discovers it. The frontmatter (`name` + `description`) is how the model decides when to fire it. Everything below the frontmatter is the instruction set.

Quickstart — the whole loop on one page

1. **Script Writer** turns one idea into a scored viral script in your voice.
2. **Reel-to-Template** steals the *structure* of any competitor's viral reel — legally.
3. **Video Builder** renders the avatar, adds captions, and composites the finished reel with one command.
4. **Thumbnail Designer** generates the scroll-stopping cover, colour-matched to your set.

5. **Caption/Post Writer** writes the IG caption, hashtags, and the comment-keyword CTA.

```
idea
  → script-writer      →  script.txt   (DSSCL ≥ 9.5)
  → reel-to-template   →  format + timing to copy   (optional)
  → video-builder      →  render → captions → composite → 1.3× + thumbnail
  → thumbnail-designer →  thumbnail.png
  → caption-writer     →  caption.txt + keyword CTA
  → POST
```

Below, each skill in full. Every `SKILL.md` block is copy-paste ready — drop it into `.claude/skills/<name>/SKILL.md` and it works.

---

1 Script Writer

SKILL · TURNS ONE IDEA INTO A SCORED VIRAL SCRIPT

You give it a single topic. It picks a proven hook, picks a proven format, writes the full script in your speaking voice, then **scores itself** and rewrites until it's genuinely postable — before you ever spend a rupee on rendering.

Two banks make this work: a **hook bank** (200 tested hook templates) and a **format bank** (47 content structures). The skill fills the hook blanks with your topic, wraps it in the GOAT framework, and grades the draft on **DSSCL**: Double-watch, Share, Save, Comment, Like. If it doesn't clear 9.5/10, it rewrites.

.CLAUDE/SKILLS/SCRIPT-WRITER/SKILL.MD

```
---
name: script-writer
description: Turn a single idea into a full viral short-form script in Anand's
  voice. Use whenever the user gives a topic, hook angle, or "write me a reel about".
---
You are the scriptwriter for @automatewithanand — AI automation for founders and
creators in India. Confident, direct, friend-to-friend. Never corporate.

WORKFLOW
1. Read reference/english-hooks.md. Pick the 2-3 best-fit hooks for the topic,
   fill the blanks with automation-specific language, choose the strongest.
2. Read reference/content-formats.md. Pick the format whose shape matches the goal.
3. Write the script: hook in the first 3 seconds (open a loop, never close it),
   no filler, one core idea, spoken 8th-grade English, build to one quotable line.
4. Score the draft on DSSCL (0-10 each):
   D Double-watch · S Share · S Save · C Comment · L Like
   final = D*.30 + Share*.25 + Save*.25 + C*.10 + L*.10
```

```
   Must clear: D>=9, Share>=9, Save>=9, final>=9.5. If not, rewrite. Max 5 tries.
5. Every script must name ONE specific tool AND end on a comment-keyword CTA.

OUTPUT: the spoken script + the DSSCL scores + the hook # and format # used.
```

Run it — open Claude Code in the project and just talk to it:

> **EXAMPLE PROMPT**
>
> Use the script-writer skill. Topic: "5 Claude Code skills that replaced my content team." Hook angle: fear of being out-produced. Keyword CTA: CLAUDE. Return the script and the DSSCL scores.

Prefer a hands-off run? The same GOAT + DSSCL loop is wired into the pipeline — dry-run it to see the scored script without spending on any API:

```
python3.11 pipeline.py --topic "5 Claude Code skills that replaced my content team" --dry-run
```

Voice trick: paste 3 of your own past captions or a transcript into the skill once and add "match this voice." The scores get honest and the writing stops sounding like an AI intern.

Pitfall 1 — one idea per script. The moment a script tries to teach two things, Save and Double-watch collapse. If DSSCL won't clear 9.5, it's almost always because the idea is two ideas.

Pitfall 2 — never skip the hook bank. A hand-written hook feels fine and dies in the feed. The bank exists because those 200 openings are already proven to stop the scroll.

2 Reel-to-Template

SKILL · REVERSE-ENGINEERS ANY VIRAL REEL INTO A REUSABLE FORMAT

See a competitor's reel doing huge numbers? This skill downloads it, pulls out evenly-spaced frames and a word-level transcript, and reduces the whole thing to a **reusable template** = layout + script pattern + timing. You're not copying their content — you're copying the *structure* that made it work, and pouring your own topic into it.

It works because Instagram exposes a public GraphQL endpoint. One `doc_id` + the shortcode returns the real `video_url` — no login. Then ffmpeg grabs the frames and ElevenLabs Scribe transcribes the audio with timestamps, so you can see exactly *when* each beat lands.

.CLAUDE/SKILLS/REEL-TO-TEMPLATE/SKILL.MD

```
---
name: reel-to-template
description: Download a competitor's Instagram reel and reduce it to a reusable
  format = layout + script pattern + timing. Use when the user shares a reel URL
  and wants to understand or replicate why it works.
---
You dissect reference reels into templates this channel can rebuild.

WORKFLOW
1. Run: python3.11 capture/analyze_reference.py
   It downloads reel.mp4, extracts frames f_00..09.png, and writes a Scribe
   transcript (words.json + transcript.txt) under
   assets/reference_analysis//.
2. Read the frames and the transcript. Produce a TEMPLATE with 3 parts:
   - LAYOUT: what's on screen each beat (full-frame face / split / card / board),
     text style, where captions sit, accent colour.
   - SCRIPT PATTERN: the sentence shape, beat by beat (hook -> reveal -> proof ->
CTA).
   - TIMING: seconds per beat, from the word timestamps. Reveals fire at
     word_start - 0.15s.
3. Map it to the closest format in formats/registry.py (run: python3.11 reel.py
   list). If none fit, describe the new layout precisely enough to build.

RULES: download IMMEDIATELY — signed video URLs expire in minutes. Never reuse the
competitor's words or footage; only the structure.
```

The command the skill runs for you:

```
python3.11 reel.py analyze https://www.instagram.com/reel/SHORTCODE/
```

Then hand the frames + transcript back to Claude with the analysis prompt:

ANALYSIS PROMPT

Use the reel-to-template skill on the files in assets/reference_analysis/<shortcode>/.
Give me: LAYOUT per beat, the SCRIPT PATTERN as a fill-in-the-blank template,
and TIMING in seconds. Then tell me which format in reel.py list is closest.

Free upgrade: if a Cloudflare wall blocks a page you want to screen-record for the same reel, pull it from the Wayback Machine (`web.archive.org/web/<year>/<url>`) instead.

Pitfall 1 — the token expires. That `video_url` is signed and dies in minutes. Download in the same run you fetch it; don't paste it into a browser "to check" first.

Pitfall 2 — copy structure, never content. Templates are legal and smart. Re-using their script lines or clips is neither. Pour your own topic into the shape.

3 Video Builder

SKILL · SCRIPT IN, FINISHED REEL OUT

This is the one that replaced the editor. It takes your script and produces the finished, posture-perfect reel: renders a talking-head **avatar** of you (HeyGen), sets the background and framing, burns in word-level **captions** (Scribe), composites the chosen **format** on top, then finishes with a **1.3× speed pass and a thumbnail end-card** — the last two are what make AI footage feel human and retention-friendly.

The trick is picking the right format first. `reel.py decide` reads your script's shape and tells you which of the format modules fits — visible-framework boards beat plain b-roll 5-10× on the same topic.

.CLAUDE/SKILLS/VIDEO-BUILDER/SKILL.MD

```
---
name: video-builder
description: Turn a finished script into a finished reel — avatar render, framing,
  captions, format compositing, 1.3x speed + thumbnail. Use when a script is ready
  to become a video.
---
You run the build line. The script is already written; you produce the MP4.

WORKFLOW (run in order)
1. Pick the format from the script shape:
   python3.11 reel.py decide ""
2. Render the avatar with a registered look (COSTS CREDITS — needs --yes):
   python3.11 reel.py render green_bookshelf_front --script-file script.txt --yes
   -> writes assets/avatar/avatar_video.mp4
3. Word-level captions via ElevenLabs Scribe:
   python3.11 reel.py transcribe assets/avatar/avatar_video.mp4 words.json
4. Composite the chosen format + captions (auto-finishes 1.3x + thumbnail):
   python3.11 reel.py make  --avatar assets/avatar/avatar_video.mp4 \
       --captions words.json
   -> writes the composited reel AND the finished reel.

RULES
- Verify the look's crop before compositing. If reel.py render prints
```

```
  "crop UNVERIFIED", extract one frame, check face position, add a preset to
  core/framing.py + core/looks.py first. A wrong crop = decapitated avatar.
- Dark pills sit behind every landed caption/label (white text drowns on the warm
set).
- End the reel the instant the last payoff lands.
```

The full command sequence, start to finish:

```
# 1. which format fits this script?
python3.11 reel.py decide "$(cat script.txt)"

# 2. render the avatar (this one spends HeyGen credits)
python3.11 reel.py render green_bookshelf_front --script-file script.txt --yes

# 3. word-level captions
python3.11 reel.py transcribe assets/avatar/avatar_video.mp4 words.json

# 4. composite + auto-finish (1.3x speed + thumbnail end-card)
python3.11 reel.py make process_walkthrough \
    --avatar assets/avatar/avatar_video.mp4 --captions words.json
```

Why 1.3×: HeyGen speaks at ~142 words/min, which reads as sluggish. The finish pass speeds video + audio to ~185 wpm — the pace top creators actually post at. It runs automatically inside `reel.py make`; use `reel.py finish <video>` to apply it to any clip manually.

Pitfall 1 — unverified crops decapitate the avatar. Every camera setup frames the face differently. Never composite on a look whose crop you haven't frame-checked once.

Pitfall 2 — `--yes` costs money. The render step spends real HeyGen credits. Lock the script (DSSCL cleared) before you render — don't render a draft you'll rewrite.

4 Thumbnail Designer

SKILL · GENERATES THE SCROLL-STOPPING COVER

The cover decides the click. This skill generates a `thumbnail.png` keyed to your avatar's background palette: a bold 3-5 word headline, your face, and a small badge — high contrast, readable at thumbnail size. The finish pass then appends it as the end-card, and you upload the same PNG as the reel cover.

.CLAUDE/SKILLS/THUMBNAIL-DESIGNER/SKILL.MD

```
---
```

```
name: thumbnail-designer
description: Design a scroll-stopping reel cover (thumbnail.png) matched to the
  avatar's set palette — bold headline + face + badge. Use after the video is built.
---
You design covers that win the click at 120px wide.

RECIPE
1. Read the reel's hook and the look's setting (core/looks.py) to get the
   background palette. green_bookshelf = warm/amber; white_darkwall = red accent;
   black_blinds = cool/moody.
2. Headline: 3-5 words, the single most curiosity-driving phrase from the hook.
   All caps, heaviest weight, one accent word in the set's accent colour.
3. Composition: face on one third, headline on the other, small handle badge
   (@automatewithanand). One accent colour only — pull it from the background.
4. Contrast: dark scrim behind text over any busy area. Test it shrunk to 120px —
   if you can't read it small, it fails.
5. Export 1080x1920 PNG to assets/final/thumbnail.png so core/finish.py appends it.

RULES: never more than 5 words. Never two accent colours. Face must show one
readable emotion (shock / conviction), not a neutral stare.
```

Prompt to fire it:

PROMPT

Use the thumbnail-designer skill. Hook: "5 Claude Code skills that replaced my content team." Look: green_bookshelf_front (warm amber set). Give me the headline (max 5 words, one accent word), the layout, and export assets/final/thumbnail.png at 1080x1920.

Palette match: pulling the accent straight from the background is what makes the cover look designed instead of slapped-on. Warm set → amber/red accent; cool set → electric blue.

Pitfall 1 — too many words. More than 5 words and nobody reads it mid-scroll. Cut until it hurts.

Pitfall 2 — a neutral face. A calm expression reads as boring. Pick the frame with real emotion.

5 Caption / Post Writer

SKILL · WRITES THE CAPTION, HASHTAGS, AND KEYWORD CTA

The last mile. This skill writes the Instagram caption in your voice, a tight hashtag set, and — the part that actually grows the account — the **comment-keyword CTA** that turns viewers into DMs.

("Comment CLAUDE and I'll send you the full guide.") Every comment also boosts the reel's reach, so the CTA does double duty.

.CLAUDE/SKILLS/CAPTION-WRITER/SKILL.MD

```
---
name: caption-writer
description: Write the Instagram caption, hashtags, and comment-keyword CTA for a
  finished reel, in Anand's voice, ready to paste. Use as the final step before
posting.
---
You write the post copy for @automatewithanand. Confident, direct, no corporate tone.

OUTPUT (in this order)
1. HOOK LINE: first line repeats the reel's promise in fresh words — this is all
   most people read before "...more".
2. BODY: 2-4 short lines of real value or a mini-list. No fluff, no "In today's
video".
3. CTA: one comment-keyword line. Format:
   "Comment  and I'll DM you the full guide." Exactly ONE keyword.
4. HASHTAGS: 5-8, mix of broad (#ai #automation) and niche
   (#claudecode #contentautomation #aivideos). No 30-tag walls.

RULES: one keyword only (two confuses the auto-DM). End on a question sometimes —
replies boost reach. Never write more than ~4 lines of body; the reel is the content.
```

The caption template it produces looks like this:

CAPTION TEMPLATE

I run my entire content business with 5 Claude Code skills. No editor. No agency.

Script → avatar video → thumbnail → caption → posted. One person, one afternoon.

The full setup — every SKILL.md and command — is in a free guide.
Comment
CLAUDE
and I'll DM it to you 👇

#ai #automation #claudecode #contentautomation #aivideos #contentcreator

Deliverability: when someone comments the keyword, reply to their comment as well as DMing the link. The reply signals a real conversation and gets your DM out of the "requests" folder.

Pitfall 1 — two keywords. If the CTA says "comment CLAUDE or GUIDE," your auto-DM tool misfires and people get nothing. Exactly one keyword, every time.

Pitfall 2 — a caption that repeats the reel. The caption should *extend* the reel (a bonus line, a question) — not transcribe it. Give a reason to read past "...more".

The 5 mistakes that stall beginners

- **Using prompts instead of skills** — if you re-explain your voice and commands every time, you never compound. Save the five `SKILL.md` files once.
- **Rendering before the script clears DSSCL** — Skill 1 is free, HeyGen credits are not. Never render a draft you'll rewrite.
- **Compositing on an unverified crop** — a wrong crop decapitates the avatar. Frame-check each look once, add the preset, then build.
- **Skipping the 1.3× finish** — un-sped AI footage feels slow and gets scrolled. The finish pass is not optional polish; it's the retention.
- **Two keywords in the CTA** — it breaks the auto-DM and kills the whole point of the funnel. One keyword, always.

This is the exact system I post with every week

Five skills, one person, zero crew. I share the real builds — screens, prompts, and the misses too — one automation every week.

Follow **@automatewithanand** on Instagram · Subscribe on YouTube · Comment **CLAUDE** on the reel if you want the next guide first.